Guía de operación del robot SCARA

ROS 2 Jazzy + micro-ROS en Ubuntu | Panel de control, motores y observadores

Juan David Botero Cubides - Mecatrónica, Universidad EIA

1. Cómo está armado el sistema



El control de bajo nivel (lazo de 1 kHz, PID, cinemática inversa, seguridad y los **observadores de Luenberger y Kalman**) corre **dentro del ESP32**, en C. ROS 2 solo envía consignas y recibe el estado y las estimaciones. El ESP32 se conecta al PC por el cable USB: el **agente micro-ROS** lo convierte en un nodo más de ROS 2. El **panel de control** (`scara_panel.py`) muestra todo en gráficas y permite manejar el robot.

2. Antes de empezar (lista de chequeo)

[ ]	ESP32 conectado por USB y placa de potencia alimentada.
[ ]	Ningún monitor serie abierto (pio device monitor, Arduino, etc.): el puerto es de uno solo.
[ ]	Robot despejado de cables y objetos, y el corte de alimentación al alcance de la mano.
[ ]	Puerto detectado: ejecuta `ls /dev/ttyUSB*` y debe aparecer /dev/ttyUSB0.
[ ]	Firmware en modo micro-ROS ya cargado (solo se repite si cambias algo del firmware; ver sección 11).
[ ]	Una sola vez: `sudo apt install python3-pyqt5 python3-pyqtgraph` (panel con gráficas).
[ ]	Archivos en `~/Trabajos/Robotica/Robot/scara_fw_microros/scara_fw/ros2/`: scara_panel.py, scara_common.py, scara_client.py, scara_teleop_ros.py, scara_rviz.py, scara_rviz.launch.py, scara.urdf, scara.rviz.

3. Paso a paso para iniciar

Abre las terminales en este orden. Las terminales nuevas ya cargan ROS 2 porque tu `~/.bashrc` hace el `source`. Deja abiertas las terminales 1 y 2 durante toda la demostración.

Terminal 1 - El agente micro-ROS (puente con el ESP32)

```
docker run -it --rm -v /dev:/dev --privileged --net=host \
    microros/micro-ros-agent:jazzy serial --dev /dev/ttyUSB0 -b 115200 -v6
```

Después pulsa el botón **EN** del ESP32 para reiniciarlo. **Qué imprime:** mientras el robot se conecta, líneas de información; ya conectado, un flujo continuo de líneas debug (es el tráfico del robot, es normal que no pare). Si eso aparece, la comunicación está viva.

```
[...] debug | SerialAgentLinux.cpp | recv_message | [==>> SER <<==] | client_key: 0x2AECB338 ...
[...] debug | DataWriter.cpp       | write        | [** <<DDS>> **]  | client_key: 0x00000001 ...
```

Terminal 2 - El panel de control

```
cd ~/Trabajos/Robotica/Robot/scara_fw_microros/scara_fw
python3 ros2/scara_panel.py
```

Qué se ve: la ventana del panel. Arriba a la izquierda debe decir ● **ESP32 conectado** (verde) y el modo (OFF al encender). Si dice sin datos del ESP32, revisa la terminal 1. Sin robot puedes practicar con `python3 ros2/scara_panel.py --demo` (simula el ESP32 con el modelo identificado).

Terminal 3 (opcional) - RViz

```
cd ~/Trabajos/Robotica/Robot/scara_fw_microros/scara_fw
ros2 launch ros2/scara_rviz.launch.py
```

Se abre RViz con la vista desde arriba, los dos círculos verdes del espacio de trabajo y el robot.

Terminal 4 - Comprobar que todo está conectado

```
ros2 node list
ros2 topic list
ros2 topic echo /scara/state --once
```

Qué imprime:

```
/scara/scara_esp32      <- el robot (nodo que corre en el ESP32)
/scara_panel
/scara_rviz  /robot_state_publisher  /rviz

/joint_states  /scara/cmd  /scara/ee_position  /scara/event  /scara/goto
/scara/jog     /scara/markers  /scara/obs  /scara/power  /scara/state

data: -0.0, 0.0, 190.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...   (19 numeros; ver la seccion 9)
```

Antes del Home el estado es q1 = 0, q2 = 0 y x = 190: no significa nada físico todavía, porque los encoders arrancan en cero al encender. **Si /scara/scara_esp32 no aparece, el agente no está conectado** (sección 10).

4. Demostración sugerida (con el panel)

El **Home es obligatorio** después de cada encendido (sin Home el firmware rechaza los movimientos).

#	Acción en el panel	Qué se ve / qué decir
1	Botón **HOME**.	Cada eje toca sus dos topes, mide su recorrido y vuelve. Home J1 ✓ y Home J2 ✓ en el encabezado.
2	Mover a una consigna: x = 120, y = 80, z = 0 → **Ir a (x, y, z)**.	La cinemática inversa se calcula en el ESP32. Evento #LLEGO con el error medido con encoders.
3	x = 400, y = 0 → Ir.	#ERROR: inalcanzable (alcance de 50 a 190 mm). Validación de consignas.
4	q1 = 20, q2 = 90 → **Ir a ángulos**.	Consigna articular. En la pestaña **Posición** se ve el brazo y la estela.
5	**Posición · flechas**: mantener ESPACIO (o MANTENER) + flechas.	Manual con dead-man: al soltar, se detiene.
6	**Potencia · deslizadores**: M1 en ▶ 30 %, mantener el dead-man.	Lazo abierto. J2 (0 %) mantiene su posición. Al soltar frena y se queda donde paró.
7	Habilitar **M3** (interruptor), deslizador ▶ 40 %, mantener el dead-man.	El atornillador gira; sus observadores aparecen en Observadores → M3.
8	Deshabilitar **M2** e Ir a ángulos.	Solo se mueve J1; M2 queda libre (se puede mover a mano y no pierde el Home).
9	Pestaña **Observadores**, M1: mover y luego **Reiniciar observadores**.	Medida vs Luenberger vs Kalman a priori/a posteriori; corriente estimada; σ y K convergen ($K_w \approx 0{,}266$).

#	Acción en el panel	Qué se ve / qué decir
10	**Grabar CSV** durante una prueba.	Archivo scara_AAAAMMDD_HHMMSS.csv con todas las estimaciones (20 Hz).

5. Qué imprime cada comando (salidas esperadas)

Los mismos comandos sirven en la línea de comandos del panel, con `python3 ros2/scara_client.py cmd "..."` o con `ros2 topic pub /scara/cmd ...`.

home

```
#OK: home solicitado
#HOME: iniciando. Eje 2: primero un tope, luego el otro, y vuelve a donde estaba.
#HOME: eje 2 referenciado en el tope MIN (q = -143.00 deg). Ahora el tope contrario...
#HOME: eje 2 recorrido medido: de -143.00 a 132.76 deg (total 275.76 deg). Limites por software: -140.02 a 129.78 deg.
#HOME: ahora el eje 1, buscando su primer tope...
#HOME: eje 1 referenciado en el tope MIN (q = -108.00 deg). ...
#HOME: terminado. Volviendo a la posicion inicial (q1 = -1.12, q2 = 0.00 deg)...
#LLEGO: q1=-1.121 q2=0.000 deg | medido x=189.96 y=-3.72 mm
```

goto 120 80 0 y goto 400 0

```
#INFO: cinematica inversa (120.0, 80.0, 0.0) -> q1 = 4.78 deg, q2 = 84.88 deg (codo +1). Moviendo...
#LLEGO: objetivo x=120.00 y=80.00 z=0.00 | medido x=120.15 y=79.73 | error ex=0.153 ey=-0.271 mm | q1=4.651 q2=84.900 deg
#ERROR: no se puede ir a (400.0, 0.0): inalcanzable (fuera del espacio de trabajo). Alcance: 50.0 a 190.0 mm.
```

Motores, observadores y otros

```
> motor 3 on      -> #OK: motor 3 habilitado          y   #MOTOR: M3 HABILITADO.
> motor 2 off     -> #OK: motor 2 deshabilitado       y   #MOTOR: M2 deshabilitado (libre).
> motors          -> #INFO: motores: M1 ON, M2 off, M3 ON
> goto 120 80     -> #ERROR: goto necesita J1 y J2 habilitados (M2 deshabilitado: usa 'motor 2 on').
> est 1           -> #INFO: M1 polos 0.250+-0.250j L=[0.2436, 0.412] | Kalman sig_i=10.0 mA sig_w=0.30 rpm
                     sig_y=0.81 rpm | V=1.000*12*duty-0.80
> luen 1 0.1 0.1  -> #OK: M1 Luenberger con polos 0.100 +- 0.100j
                     #EST: M1 Luenberger polos 0.100 +/- 0.100j -> L = [0.036732, 0.71198]
> kal 1 10 0.3 0.81 -> #OK: M1 Kalman sig_i=10.0 mA sig_w=0.30 rpm sig_y=0.81 rpm
> est reset       -> #OK: observadores reiniciados
> gain            -> #INFO: J1 kp=5350 ki=18500 kd=295 kff=85 fric=70 | J2 kp=5000 ki=3000 kd=180 kff=80 fric=70
> stop / off / clear / status  (como antes; 'status' ahora incluye  motores:12-)
(stop con el dead-man de potencia presionado) -> #INFO: potencia manual bloqueada: suelta el dead-man ...
```

6. El panel de control (scara_panel.py)

Se abre con `python3 ros2/scara_panel.py` (`--demo` sin robot). Reemplaza al panel Tk; `scara_teleop_ros.py` queda como respaldo liviano.

Zona	Qué hace
Encabezado	Conexión con el ESP32, modo, Home J1/J2, potencia manual activa, corriente estimada de la fuente, falla y los 4 finales de carrera. Botones HOME, DETENER, BORRAR FALLA y APAGAR.
Tarjetas de motor	Interruptor para **habilitar/deshabilitar**; velocidad y corriente del motor (Kalman); PWM real; **deslizador de potencia de 0 a 100 %** con sentido ◀ ▶; debajo, el % de PWM que se envía. Clic en el valor = 0.
Control manual	**Posición** (flechas → /scara/jog) o **Potencia** (deslizadores → /scara/power). Dead-man: botón verde MANTENER o ESPACIO. Los botones de flecha también funcionan manteniéndolos.
Mover a una consigna	x, y, z en mm (codo +1 / −1) o q1, q2 en grados. En Posición, un clic en el plano elige (x, y).
Pestaña Observadores	Por motor: velocidad medida, Luenberger, Kalman a priori y a posteriori; corriente estimada; incertidumbre √P; ganancia de Kalman. Botón Reiniciar observadores.
Pestaña Motores	Velocidad, corriente, PWM y voltaje del modelo de los 3 motores juntos, y la corriente que debería marcar la fuente.

Zona	Qué hace
Pestaña Posición	Vista superior: espacio de trabajo, brazo, estela y consigna; q contra q de referencia.
Pestaña Sintonía	PID por eje (leer/aplicar), polos del Luenberger y σ del Kalman por motor, mapa de voltaje V = g·12·duty − v0 (valores del L298N del laboratorio o del TB6612), escala de los deslizadores por motor (arranque y tope) y corriente de la parte lógica.
Registro	Eventos con colores (verde #LLEGO/#OK, rojo #ERROR/#FALLA, ámbar #MOTOR/#EST/#HOME), línea de comandos con historial, ventana de tiempo, pausa y Grabar CSV.
Seguridad	Escribir en un cuadro o perder el foco suelta el dead-man. Clic fuera de los cuadros (o Esc) devuelve el teclado al robot.

Tecla	Acción	Tecla	Acción
ESPACIO (mantener)	dead-man	h	Home
← →	J1 − / +	s · Esc	Detener
↓ ↑	J2 − / +	o	Apagar motores
r	Reiniciar observadores	c	Borrar falla

7. Motores y potencia manual

Regla	Detalle
Habilitar / deshabilitar	Interruptor del panel, `motor j on|off` o `motors a b c`. Al encender: M1 y M2 habilitados, M3 deshabilitado.
Motor deshabilitado	Driver libre (sin PWM). El encoder sigue contando: no pierde el Home. No entra al Home, al PID ni a la supervisión de fallas.
goto / ang	goto necesita J1 y J2 habilitados. ang mueve solo los ejes habilitados.
Potencia manual	Solo mientras llega el dead-man (25 Hz; sin latidos 300 ms → 0). Ejes con potencia: lazo abierto. Ejes con 0 %: mantienen posición. Al soltar: frenan y se quedan donde paran.
Atornillador (M3)	Solo se mueve con potencia manual; funciona en paralelo al brazo, incluso con el brazo en OFF.
Escala 0–100 %	0 % = parado (mantiene posición); 1 % = arranque (PWM con el que apenas gira); 100 % = tope del motor. Por defecto: arranque 7 %; tope 60 % M1, 30 % M2, 60 % M3. Se cambian en Sintonía y el panel los recuerda. Por /scara/power sigue viajando % de PWM real.
Límites	Tope 63 % de PWM, rampa de 0,34 s, sin empujar contra un final de carrera, límites por software con Home.
Corriente de la fuente	La de las tarjetas es la del motor. La fuente marca $I_{lógica} + \Sigma\,\lvert i_{motor}\rvert \cdot duty$: la lógica (ESP32, reguladores, encoders, driver) se mide con los motores APAGADOS y se escribe en Sintonía; con PWM, el motor solo toma corriente de la fuente el tiempo encendido.
Bloqueo	Detener, Apagar o una falla bloquean la potencia hasta soltar y volver a presionar el dead-man.

8. Observadores (Luenberger y Kalman)

Corren en el ESP32 para los 3 motores a 100 Hz (Ts = 10 ms), con el modelo identificado en el Laboratorio 4. Miden solo la velocidad (diferencia de cuentas del encoder en 10 ms) y estiman velocidad y corriente.

```
La di/dt = V - Ra i - Kv w      J dw/dt = Kv i - B w      y = w   (eje de salida)
Luenberger:  x^(k) = A x^(k-1) + B u(k-1) + L [ y(k-1) - C x^(k-1) ]       L por Ackermann
Kalman:      x-(k) = A x+(k-1) + B u(k-1),  P- = A P+ A' + Q,  K = P- C'(C P- C' + R)^-1,
             x+(k) = x- + K (y(k) - C x-),  P+ = (I - K C) P-
```

Parámetro	Valor por defecto	Cambiar en vivo
Modelo	Ra = 50 Ω, La = 0,060 H, Kv = 0,1671 V·s/rad, B = 2,79·10⁻³, J = 3,63·10⁻⁴ (eje de salida)	config.h (MOT_*)
Luenberger	polos 0,25 ± 0,25j → L = [0,2436; 0,4120]	`luen j re im`
Kalman	σi = 10 mA, σw = 0,30 rpm, σy = 0,81 rpm (R = 0,655 rpm²) → K_w ≈ 0,266	`kal j σi σw σy`
Voltaje	V = g·(12 V·duty) − v0; TB6612: g = 1, v0 = 0,8 V · L298N (lab): g = 0,9782, v0 = 1,197 V	`vmap g v0`

Cómo leer las gráficas: el Kalman a posteriori (cian) sigue a la medida sin el ruido de cuantización; el a priori (naranja) es lo que dice solo el modelo; el Luenberger (verde) es más ruidoso en la corriente. Tras Reiniciar observadores, σ arranca alta y baja, y K_w se estabiliza en ≈ 0,266 (en estado estable el filtro le cree ~27 % a la medida y ~73 % al modelo). Con Ra = 50 Ω la corriente estimada casi no reacciona a la carga: el Kalman casi no la corrige (K_i ≈ 0).

9. Qué hace cada nodo

Nodo	Dónde corre	Qué hace	Publica / muestra
/scara/scara_esp32	ESP32	Recibe /scara/goto, /scara/jog, /scara/power y /scara/cmd; IK, control en lazo cerrado, motores habilitados, potencia manual, finales de carrera, dead-man y observadores.	/scara/state, /scara/obs, /scara/ee_position, /scara/event
micro_ros_agent	PC (Docker)	Puente: traduce lo que el ESP32 envía por USB a tópicos de ROS 2, y al revés.	Líneas debug
scara_panel	PC (Python)	Panel PyQt5: gráficas, motores, dead-man (25 Hz), consignas, sintonía, CSV.	La ventana
scara_client	PC (Python)	Un comando por ejecución; espera el evento final.	Eventos
scara_teleop	PC (Python)	Panel Tk anterior (respaldo, sin gráficas).	Registro
scara_rviz / rviz2	PC	Convierte /scara/state en /joint_states y dibuja el espacio de trabajo y la estela.	RViz

Tópicos

Tópico	Tipo	Sentido	Contenido
`/scara/goto`	geometry_msgs/Point	PC → ESP32	Consigna x, y, z en mm (z = altura fija).
`/scara/jog`	geometry_msgs/Vector3	PC → ESP32	25 Hz: x = dead-man, y = dir J1, z = dir J2.
`/scara/power`	Float32MultiArray	PC → ESP32	**Nuevo.** 25 Hz: [dead-man, %M1, %M2, %M3].
`/scara/cmd`	std_msgs/String	PC → ESP32	home, stop, off, clear, goto, ang, elbow, gain, pulse, motor, motors, est, luen, kal, vmap, status, help.
`/scara/state`	Float32MultiArray	ESP32 → PC	19 números a 20 Hz (tabla de abajo).
`/scara/obs`	Float32MultiArray	ESP32 → PC	**Nuevo.** 38 números a 20 Hz: observadores de los 3 motores.
`/scara/ee_position`	geometry_msgs/Point	ESP32 → PC	Posición medida del efector en mm.
`/scara/event`	std_msgs/String	ESP32 → PC	#OK, #INFO, #HOME, #LLEGO, #ERROR, #FALLA, #MOTOR, #EST.

Los 19 datos de /scara/state

Pos.	Significado	Pos.	Significado
0, 1	q1, q2 en grados	11, 12	PWM de M1, M2
2, 3	x, y del efector en mm	13	Motores habilitados (bits: 1 = M1, 2 = M2, 4 = M3)
4	Modo: 0 OFF, 1 HOMING, 2 HOLD, 3 MOVE, 4 JOG, 5 PULSE, 6 FAULT, 7 POWER	14	PWM de M3
5	Falla: 0 ninguna, 1 seguimiento, 2 trabado, 3 final de carrera, 4 tiempo de Home, 5 MIN y MAX	15	Potencia manual activa
6, 7	Home hecho en J1, J2	16, 17, 18	Velocidad medida M1, M2, M3 [rpm]
8	Finales de carrera (bits): 1 J1 min, 2 J1 max, 4 J2 min, 8 J2 max		
9, 10	Referencia interna de q1, q2 en grados		

/scara/obs: [secuencia, banderas, 12 datos por motor × 3]

Por motor: velocidad medida, Luenberger, Kalman a priori, Kalman a posteriori [rpm]; corriente Luenberger, Kalman a priori, Kalman a posteriori [A]; σ corriente [A]; σ velocidad [rpm]; K_w; K_i; voltaje del modelo [V]. Banderas: bit m = diseño válido del motor m; bit m+3 = el motor tiene encoder.

10. Errores frecuentes y cómo resolverlos

Síntoma / mensaje	Causa probable	Solución
El panel dice 'sin datos del ESP32' / No llega estado	El agente no corre, puerto o baudios distintos, monitor serie abierto, o el ESP32 no se reinició.	Revisa la terminal 1. Cierra monitores. Confirma -b 115200 y /dev/ttyUSB0. Pulsa EN.
Falta PyQt5 o pyqtgraph	No están instaladas las librerías del panel.	`sudo apt install python3-pyqt5 python3-pyqtgraph`
docker: command not found / permission denied	Docker no instalado o usuario fuera del grupo docker.	`sudo apt install docker.io`, `sudo usermod -aG docker $USER` y reinicia la sesión.
#ERROR: primero ejecuta HOME	Se encendió el robot o se borró una falla.	Pulsa HOME.
#ERROR: goto necesita J1 y J2 habilitados	Un motor del brazo está deshabilitado.	Actívalo con su interruptor o `motor 2 on`.
#INFO: potencia manual bloqueada	Detener/Apagar/falla con el dead-man presionado.	Suelta el dead-man y vuelve a presionarlo.
M3 no gira con potencia	M3 arranca deshabilitado, o no estás en modo Potencia.	Interruptor de M3, modo Potencia y mantener el dead-man.
Velocidad medida de M3 siempre 0	Pines del encoder 3 (J8) distintos a los de config.h.	Corrige PIN_ENC3_A/B o pon M3_TIENE_ENCODER 0.
#FALLA: final de carrera inesperado al iniciar Home	El brazo quedó apoyado en un tope o un interruptor da señal falsa.	Apagar, mover el brazo al centro, mirar los topes en el encabezado (todos apagados), Borrar falla y Home.
#FALLA: error de seguimiento o **motor trabado**	Algo frena el motor, signo invertido o ganancias.	Libera el brazo, Borrar falla, repite Home. Si vuelve: pulse 1 / pulse 2 y revisa ganancias (respaldo: 5000 3000 180 80 70).
#FALLA: tiempo de Home agotado	PWM de búsqueda bajo o brazo trabado.	Revisa que nada estorbe; sube HOME_DUTY (170) en config.h.
#ERROR: no se puede ir a (x, y): inalcanzable	Fuera de 50 a 190 mm o singular.	Punto dentro de los círculos o el otro codo.

Síntoma / mensaje	Causa probable	Solución
Un eslabón muy agresivo (vibra, busca la posición, PWM alto estando quieto)	Ganancias altas para ese eje (sobre todo ki).	Sintonía: baja ki y luego kd, Aplicar (dura hasta reiniciar). Para dejarlo fijo: src/config.c. J2 ya viene con 5000 3000 180 80 70.
#ERROR: no llego a la consigna a tiempo	El brazo se detuvo justo fuera de la tolerancia (fricción).	Revisa ganancias (fric, ki) o repite.
El robot se detiene a ratos en manual	Se pierden latidos del dead-man (300 ms).	Comportamiento seguro; cierra programas pesados, revisa el USB.
Failed to resolve component 'micro_ros_espidf_component'	El zip no trae el componente micro-ROS (es otro repositorio).	git clone -b jazzy https://github.com/micro-ROS/micro_ros_espidf_component.git components/micro_ros_espidf_component
Las gráficas van 'a saltos'	Ancho de banda a 115200 baudios.	Sube a 460800: `uros_select.py serial --baud 460800` y -b 460800 en el agente.
RViz: el robot no aparece / ejes gigantes	scara_rviz no corre / display TF activo.	ros2 node list debe mostrar /scara_rviz; desmarca TF.
`Could not exclusively lock port`	Otro programa tiene el puerto.	`fuser -v /dev/ttyUSB0` y cierra ese proceso.

11. Si hay que volver a compilar el firmware

Solo hace falta si cambias `config.c` o `config.h` (pines del encoder 3, motores habilitados al arrancar, ganancias, modelo, observadores). En una carpeta nueva (zip o clon) primero hay que clonar el componente micro-ROS, que no viene incluido, y elegir el modo (esto también lo deja compatible con ESP-IDF 5.3):

```
git clone -b jazzy https://github.com/micro-ROS/micro_ros_espidf_component.git components/micro_ros_espidf_component
python3 tools/uros_select.py serial
```

Compila siempre con un entorno limpio, sin ROS cargado:

```
env -i HOME="$HOME" TERM="$TERM" \
  PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:$HOME/.local/bin" \
  bash --noprofile --norc -c \
  'cd ~/Trabajos/Robotica/Robot/scara_fw_microros/scara_fw && pio run -t upload'
```

Error al compilar	Solución
`Unknown boolean field "use_gitignore"`	`python3 tools/uros_select.py serial` lo corrige solo; a mano: `sed -i '/use_gitignore/d' components/micro_ros_espidf_component/idf_component.yml`
`CMake 3.20 or higher is required`	Enlazar el CMake del sistema: `ln -s /usr/bin/cmake ~/.platformio/packages/tool-cmake/bin/cmake` (después de mover el original a cmake.orig).
`Could not find ament_cmake` / errores raros de ROS	Compilaste con ROS cargado: usa el comando env -i de arriba.
`*.obj: No such file` en micro_ros	Borra components/micro_ros_espidf_component/{micro_ros_dev,micro_ros_src,libmicroros.a,include} y compila de nuevo.
Se reinicia solo al conectar	Sube CONFIG_SCARA_UROS_STACK (menuconfig).

12. Al terminar

Cierra con Ctrl+C (o la ventana) en este orden: el panel, RViz y por último el agente (terminal 1). Los motores quedan en HOLD; si cortas la alimentación de potencia se sueltan. Al volver a encender hay que repetir el Home.